

3D 차량 충돌 물리 시뮬레이터

(3D Car Collision Simulator)

프로젝트 보고서

프로젝트 구분:	3D WebGL 기반 가상 시뮬레이션 Sandbox
개발 도구 (GenAI):	Google AI Studio Build (Antigravity Agent + Gemini 3.5 Flash Model)
개발 언어 & 스택:	TypeScript, React 19, Three.js, Recharts, Tailwind CSS
GitHub 저장소:	boratown/CAR-COLLISION-SIMULATOR
보고서 작성일:	2026년 7월 16일

1. 프로젝트 기획 의도

- 물리 기반 인터랙티브 학습 및 실험 환경 제공: 수학 및 물리적 개념(속도 변화, 질량 비율, 강체 충돌 에너지 전이 등)과 차량 파손 거동을 브라우저상에서 직접 튜닝하며 시각적으로 배울 수 있는 가상 실험실을 구축하고자 하였습니다.
- 충돌 안전 지표 수치화: 정면, 측면, 후면 등 정교한 충돌 시나리오 프리셋을 통해 충돌 순간의 감속도(G-Force), 충돌에너지(kN), 그리고 탑승자 더미(Occupant Dummy)의 부상 위험 지표(HIC, Chest G)를 실시간 그래프로 표현하여 안전 설계의 중요성을 체감하게 만듭니다.
- 파손 시각화: 차량 외장에 가해지는 가속도와 충격 방향에 따라 범퍼, 사이드미러, 램프, 스포일러, 보닛 등의 외장 메쉬가 유기적으로 찌그러지거나 이탈하도록 고안되어 역동적인 충돌 역학을 사실감 있게 묘사합니다.

2. 전체 아키텍처

A. 메뉴 구성 (Menu Configuration)

- 3D 충돌 테스트베드 (WebGL Sandbox): 실시간 3D 뷰포트를 통해 마찰력, 카메라 뷰포트 변경(탑승자 1인칭, 자유 궤도 조작) 및 차량의 충돌 현장을 실시간으로 확인합니다.
- 시뮬레이션 컨트롤 패널 (Control Panel): 정면충돌, 측면 T-본(T-Bone) 충돌, 후방 충돌, 다중 충돌 시나리오 프리셋을 선택하고 시뮬레이션의 물리적 변수(충돌 탄성, 노면 마찰력, 시간 배율) 및 차량 스펙(속도, 중량, 브레이크 성능 등)을 즉각적으로 수정합니다.
- 실시간 텔레메트리 (Telemetry Dashboard): 충돌 직후의 충돌 속도, 감속 비율, 실시간 G-Force 타임라인 차트와 더미 부상 데이터를 제공합니다.

B. Dataflow & Workflow

1. 설정 단계 (Input): 사용자가 컨트롤 패널에서 충돌 속도, 질량, 탄성 등을 튜닝하거나 시나리오 프리셋을 선택합니다.
2. 물리 프레임 루프 (Physics loop & SAT): Three.js 프레임 루프가 60FPS로 돌며 차량 경계 상자(OBB)의 충돌 여부를 분리축 정리(SAT, Separating Axis Theorem) 솔버로 탐지하고 임펄스를 역학적으로 해결합니다.
3. 이벤트 트리거 (Breakable Part System): 충격량(kN)이 부품 한계치를 초과하고 충격 포인트가 해당 부품 영역(전/후/좌/우)에 해당할 시, 부품 메쉬가 차체 그룹에서 해제되어 월드 공간에 물리 전이와 함께 이탈합니다. (바퀴 이탈 및 복잡한 파티클 이펙트는 안전한 시뮬레이션 흐름과 성능을 위해 최적화되었습니다.)
4. 시각화 갱신 (Telemetry Update): 시뮬레이터의 물리 데이터 프레임이 React 상태(History Array)로 연속 전송되어, Recharts 라이브러리로 수렴되어 그래프를 실시간 플로팅합니다.

3. 사용 코딩 도구(GenAI) 및 프롬프트 예시

● 코딩 도구: Google AI Studio Build (Antigravity Agent + Gemini 3.5 Flash Model)

● 프롬프트 1: "차량 충돌 3D 시뮬레이션 기능을 갖춘 물리 엔진 기반의 앱을 만들어줘"

-> (적용 내용: React, TypeScript, Three.js(WebGL)를 기반으로 차량 충돌 테스트베드 3D 뷰포트, 물리 엔진 솔버(SAT 강체 충돌), 환경 제어 패널 및 실시간 텔레메트리 그래프를 갖춘 시뮬레이터 프로젝트를 초기 설계 및 구축함)

● 프롬프트 2: "바퀴 파손을 제거 충돌 파티클 제거"

-> (적용 내용: 차체 지탱과 주행의 핵심 요소인 4바퀴에 대한 파손 및 이탈 메커니즘을 영구 제거하여 충돌 시에도 바퀴가 분리되지 않고 지면을 유지하게 개선하고, 가볍거나 잦은 충돌에서 성능을 저해하고 비현실적이던 스파크, 연기, 유리 파편 파티클 시스템을 제거하여 쾌적한 비주얼 연산을 도모함)

● 프롬프트 3: "가벼운 충격에도 모든 외장 부품이 사방으로 날아가던 현상을 보완해줘"

-> (적용 내용: 파손에 필요한 힘의 임계값(Threshold)을 크게 높이고, 충돌 위치 좌표(Z축/X축)와 부품 위치가 정밀하게 일치하는 사분면 영역 타격 검증식을 더해 현실적인 부분적 파손이 일어나도록 재조정함)

4. 기술 스택 및 결과물 MVP

A. 기술 스택 (Tech Stack)

- 언어 & 빌드 도구: TypeScript, Vite
- 프레임워크 & UI: React 19, Tailwind CSS, Lucide React (아이콘 라이브러리)
- 3D 그래픽 엔진: Three.js, WebGL (OrbitControls, 2.5D Euler-Verlet 물리 모델, SAT 임펄스 강체 계산)
- 실시간 차트: Recharts (실시간 물리 텔레메트리 데이터 플로팅)

B. 최종 결과물 MVP

- Vercel 배포 URL: My Google AI Studio App
- GitHub 리포지토리: boratown/CAR-COLLISION-SIMULATOR

5. 프로젝트 소감 (PMI)

● Plus (강점 & 긍정적인 면)

- 무거운 물리 엔진 외부 라이브러리 없이 순수 Three.js 상에서 SAT 솔버와 오일러-베를레(Euler-Verlet) 수식의 차량 강체 충돌을 완전 무결하게 구현했습니다.
- 물리 법칙 데이터(G-Force, HIC 등)가 실시간 동적 차트로 부드럽게 흐르며 차트와 시뮬레이터가 유기적으로 호흡하여 인터랙티브 교육 도구로서 우수한 활용도를 지닙니다.
- 최근 적용한 파손 완화 수식으로 인해 한층 묵직하고 실물에 가깝게 부서지는 자연스러운 거동을 느낄 수 있습니다.

● Minus (보완점 & 아쉬운 면)

- 온전한 3D 전복(Rollover) 물리 효과나 지형 경사면 반영이 아닌 수평 지면 위에서의 2.5D 평면 물리 좌표계 기반으로 회전 및 임펄스를 계산하여 높낮이에 의한 입체적 충격 굴절에는 한계가 있습니다.
- 메쉬를 물리적으로 직접 조각내서 자르는 실시간 변형 기법(CSG 또는 Vertex 셰이더 조작) 대신 미리 약속된 컴포넌트(breakable parts)들을 충격에 의해 분리시키는 기법을 사용하여, 아주 세밀한 차체 철판 찌그러짐 표현까지는 단순화되어 있습니다.

● Interest (흥미로웠던 점)

- AI 스튜디오의 멀티 파일 툴킷과 GenAI의 복잡한 물리 계산 추론을 적극 융합해, 게임 엔진 없이 브라우저 단독으로 구현하기 힘든 3D 물리 시뮬레이터 샌드박스를 완벽히 조립할 수 있었다는 점이 가장 놀라웠습니다.
- 사용자의 간소화 피드백("바퀴 파손 제거 및 파티클 최적화")에 맞춰 코드의 3D 씬 정리(Cleanup) 및 최적화가 유연하게 즉각 수용되어 브라우저 성능을 획기적으로 올릴 수 있었던 최적화 과정이 고도로

인상적이었습니다.